# Attention Is All You Need

**论文基本信息**

- authors: Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin
- affiliation: Google Brain, Google Research, University of Toronto
- venue: 31st Conference on Neural Information Processing Systems (NeurIPS 2017) - source: [arXiv:1706.03762v7](arXiv:1706.03762v7)

# TL;DR

本论文提出了Transformer模型，一种完全基于自注意力（self-attention）机制、无需循环（RNN）或卷积（CNN）结构的序列建模方法。Transformer极大提升了训练并行性及性能，并在机器翻译等任务上获得了卓越的效果，定义了大规模预训练语言模型的基础架构。

# 引言

序列到序列（sequence transduction）任务（如机器翻译、摘要等）传统依赖RNN（LSTM/GRU）或CNN架构，尽管取得了进步但依然受限于串行计算、长距离依赖建模能力不足等问题。近年来，注意力机制成为关键，使模型能够忽略符号间距离直接建模全局依赖性。本文提出Transformer模型——完全弃用递归和卷积，纯粹依赖多头自注意力（multi-head self-attention）机制处理输入输出间的全局依赖，同时极大提升训练并行性和效率，成为序列建模领域的重要突破。

# 方法

## Transformer 架构总览

Transformer由编码器（Encoder）和解码器（Decoder）两部分组成，分别堆叠若干相同的子层。每层主要由多头注意力机制和前馈神经网络（Feed-Forward Netwoks）组成，均配有残差连接和层归一化。

**模型结构如图所示：**

*Transformer整体架构，左侧为编码器，右侧为解码器，均由多头自注意力和前馈子层堆叠，末端输出概率分布*

### Encoder & Decoder 堆叠层

- 编码器：6层，每层包含多头自注意力和前馈网络。
- 解码器：6层，每层包含掩码多头自注意力、编码器—解码器多头注意力及前馈网络。掩码实现自回归生成。

### 残差连接与归一化

每个子层输出都进行层归一化，并与输入作残差相加，缓解深层网络训练困难。

## Attention 机制详解

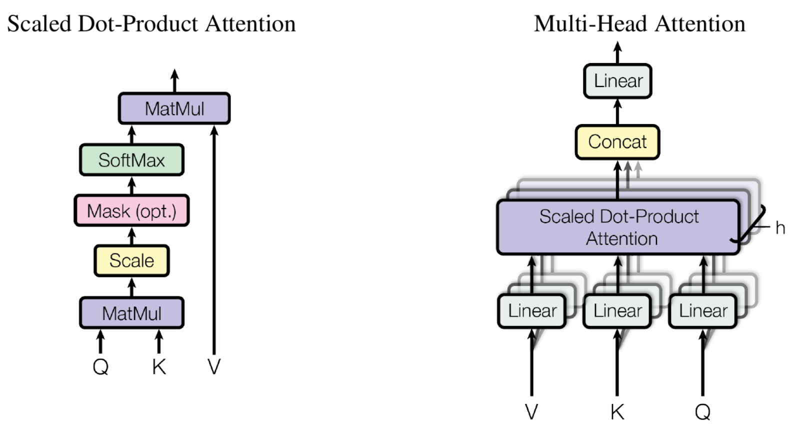
### 缩放点积注意力（Scaled Dot-Product Attention）

公式如下：

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 $Q,K,V$ 为 query, key, value，$d_k$为key的维数。

图解：




*左：缩放点积注意力流程，右：多头注意力是多个注意力并行计算后拼接*

### 多头注意力（Multi-Head Attention）

将 $Q,K,V$ 映射到不同子空间并行计算，然后拼接+线性变换。即：

$$\mathrm{MultiHead}(Q,K,V)=\mathrm{Concat}(\mathrm{head}_1,...,\mathrm{head}_h)W^O \text{where } \mathrm{head}_i=\mathrm{Attention}(QW_i^Q,KW_i^K,VW_i^V)$$

这样模型能关注不同表示子空间，不同位置细粒度的信息。

#### 应用

- Encoder-decoder attention：解码器每层查询编码器全部输出，建模输入-输出相关性。
- Self-attention：编码器或解码器内部单层自我关注建模全局依赖。

### 前馈网络

每个注意力子层后接独立、位置无关的前馈网络：

$$FFN(x)=\max(0,xW_1+b_1)W_2+b_2$$

### 嵌入与位置编码

由于Transformer结构无递归或卷积，需引入位置编码（Positional Encoding）提供序列中token顺序信息。具体采用sine/cosine函数构建，按不同频率编码绝对位置信息：

$$PE_{(pos,2i)}=\sin(pos/10000^{2i/d_{model}})PE_{(pos,2i+1)}=\cos(pos/10000^{2i/d_{model}})$$

### 不同网络结构的复杂度对比

表格1总结了自注意力、循环、卷积各自的理论计算复杂度和最大路径长度：

| Layer Type | Complexity per Layer | Sequential Operations | Maximum Path Length |
|---|---|---|---|
| Self-Attention | $O(n^2 \cdot d)$ | $O(1)$ | $O(1)$ |
| Recurrent | $O(n \cdot d^2)$ | $O(n)$ | $O(n)$ |
| Convolutional | $O(k \cdot n \cdot d^2)$ | $O(1)$ | $O(log_k(n))$ |
| Self-Attention (restricted) | $O(r \cdot n \cdot d)$ | $O(1)$ | $O(n/r)$ |

*不同类型序列模型层的复杂度与最长依赖路径对比,强调Transformer自注意力的并行性和长依赖建模能力*

## 实验

### 任务与数据

1. **机器翻译**：WMT 2014英德（EN-DE）、英法（EN-FR）大规模公开数据集。

- 训练参考量级百万对
- 评价指标：BLEU得分，训练复杂度（FLOPs）

## 主要实验设置

- 对比对象包括ByteNet、GNMT（Google神经翻译）、ConvS2S等主流模型；
- 主要实验在8块NVIDIA P100 GPU上进行，比传统RNN/CNN模型训练速度显著提升。

## 结果与性能分析

### 核心结果

| Model | BLEU | | Training Cost (FLOPs) | |
|---|---|---|---|---|
| | EN-DE | EN-FR | EN-DE | EN-FR |
| ByteNet [18] | 23.75 | | | |
| Deep-Att + PosUnk [39] | | 39.2 | | $1.0 \cdot 10^{20}$ |
| GNMT + RL [38] | 24.6 | 39.92 | $2.3 \cdot 10^{19}$ | $1.4 \cdot 10^{20}$ |
| ConvS2S [9] | 25.16 | 40.46 | $9.6 \cdot 10^{18}$ | $1.5 \cdot 10^{20}$ |
| MoE [32] | 26.03 | 40.56 | $2.0 \cdot 10^{19}$ | $1.2 \cdot 10^{20}$ |
| Deep-Att + PosUnk Ensemble [39] | | 40.4 | | $8.0 \cdot 10^{20}$ |
| GNMT + RL Ensemble [38] | 26.30 | 41.16 | $1.8 \cdot 10^{20}$ | $1.1 \cdot 10^{21}$ |
| ConvS2S Ensemble [9] | 26.36 | **41.29** | $7.7 \cdot 10^{19}$ | $1.2 \cdot 10^{21}$ |
| Transformer (base model) | 27.3 | 38.1 | $\mathbf{3.3 \cdot 10^{18}}$ | |
| Transformer (big) | **28.4** | **41.8** | $2.3 \cdot 10^{19}$ | |

*不同模型在机器翻译任务上的BLEU得分和计算量对比，Transformer在EN-DE和EN-FR上均大幅领先，且计算成本极低*

- Transformer（big配置）在英德任务上BLEU高达28.4，远超此前方法
- 在英法任务上也达到41.8。计算成本仅为GNMT等方法的1/4甚至更低

### 消融与变体实验

实验还考察了层数、注意力头数、前馈层维度、位置编码方式等对性能的影响：

perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

| | $N$ | $d_{\text{model}}$ | $d_{\text{ff}}$ | $h$ | $d_k$ | $d_v$ | $P_{drop}$ | $\epsilon_{ls}$ | train steps | PPL (dev) | BLEU (dev) | params $\times 10^6$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| base | 6 | 512 | 2048 | 8 | 64 | 64 | 0.1 | 0.1 | 100K | 4.92 | 25.8 | 65 |
| (A) | | | | 1 | 512 | 512 | | | | 5.29 | 24.9 | |
| | | | | 4 | 128 | 128 | | | | 5.00 | 25.5 | |
| | | | | 16 | 32 | 32 | | | | 4.91 | 25.8 | |
| | | | | 32 | 16 | 16 | | | | 5.01 | 25.4 | |
| (B) | | | | | 16 | | | | | 5.16 | 25.1 | 58 |
| | | | | | 32 | | | | | 5.01 | 25.4 | 60 |
| (C) | 2 | | | | | | | | | 6.11 | 23.7 | 36 |
| | 4 | | | | | | | | | 5.19 | 25.3 | 50 |
| | 8 | | | | | | | | | 4.88 | 25.5 | 80 |
| | | 256 | | | 32 | 32 | | | | 5.75 | 24.5 | 28 |
| | | 1024 | | | 128 | 128 | | | | 4.66 | 26.0 | 168 |
| | | | 1024 | | | | | | | 5.12 | 25.4 | 53 |
| | | | 4096 | | | | | | | 4.75 | 26.2 | 90 |
| (D) | | | | | | | 0.0 | | | 5.77 | 24.6 | |
| | | | | | | | 0.2 | | | 4.95 | 25.5 | |
| | | | | | | | | 0.0 | | 4.67 | 25.3 | |
| | | | | | | | | 0.2 | | 5.47 | 25.7 | |
| (E) | positional embedding instead of sinusoids | | | | | | | | | 4.92 | 25.7 | |
| big | 6 | 1024 | 4096 | 16 | | | 0.3 | | 300K | **4.33** | **26.4** | 213 |

development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

In Table 3 rows (B), we observe that reducing the attention key size $d_k$ hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [9], and observe nearly identical results to the base model.

### 6.3 English Constituency Parsing

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [37].

We trained a 4-layer transformer with $d_{model} = 1024$ on the Wall Street Journal (WSJ) portion of the Penn Treebank [25], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkleyParser corpora from with approximately 17M sentences [37]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we



*网络不同参数配置对性能影响，发现增大d_ff/h有助于提升BLEU；*
*sinusoid和learned positional encoding无显著差别*

## 英文成分句法分析（迁移学习能力验证）

4: The Transformer generalizes well to English constituency parsing (Results are on Sect
SJ)

| Parser | Training | WSJ 23 F1 |
|---|---|---|
| Vinyals & Kaiser el al. (2014) [37] | WSJ only, discriminative | 88.3 |
| Petrov et al. (2006) [29] | WSJ only, discriminative | 90.4 |
| Zhu et al. (2013) [40] | WSJ only, discriminative | 90.4 |
| Dyer et al. (2016) [8] | WSJ only, discriminative | 91.7 |
| Transformer (4 layers) | WSJ only, discriminative | 91.3 |
| Zhu et al. (2013) [40] | semi-supervised | 91.3 |
| Huang & Harper (2009) [14] | semi-supervised | 91.3 |
| McClosky et al. (2006) [26] | semi-supervised | 92.1 |
| Vinyals & Kaiser el al. (2014) [37] | semi-supervised | 92.1 |
| Transformer (4 layers) | semi-supervised | 92.7 |
| Luong et al. (2015) [23] | multi-task | 93.0 |
| Dyer et al. (2016) [8] | generative | 93.3 |

*Transformer在英文成分句法分析任务上，表现与主流方法接近甚至更优*

# 总结与讨论

## 总结

- 提出了一种完全基于自注意力机制、去除RNN和CNN的Transformer模型，大幅提升了序列建模效率和效果。
- Transformer在大规模机器翻译任务上达到当时新SOTA，并验证了极强的迁移泛化能力。
- 核心是依赖多头注意力机制实现高效全局依赖建模，极大促进大规模预训练语言模型的发展。

## 局限性与未来方向

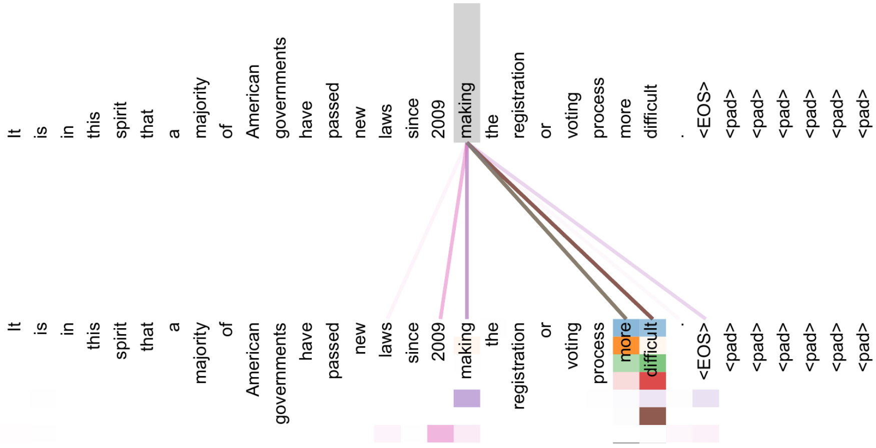
- 由于自注意力机制对序列长度的二次复杂度，极长序列处理上存在瓶颈；论文提出后续可通过局部注意力等方法改进。
- Transformer启发了更广泛的多模态（vision、audio）等领域，并为GPT、BERT等预训练模型打下基础。
- 未来可探讨高效变种、稀疏注意力、大规模多模态任务的泛化。

# 相关文献与实现

- 代码开源：[https://github.com/tensorflow/tensor2tensor](https://github.com/tensorflow/tensor2tensor)

**图示：部分关键可视化与注意力行为示例**

## Attention Visualizations




*模型中多头注意力分配实例，可解释不同头关注不同语句成分与远依赖*

以上即为本论文的简报。如需更详细数学推导、完整实验细节请参考原文。